CS214, Spring 2026,
Coding Assignment #0

Due: 11:59pm, Mon 04/06, 2026

Deadline. The homework is due **11:59pm, Mon 04/06, 2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools. If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.**

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here `https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/`

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

This is the first coding assignment. It is a very simple tutorial about 1) using course server, 2) running parallel code and 3) adding granularity control to improve performance.

Note that the deadline is 11:59pm, Mon 04/06, 2026. Please do not refer to the deadline on Github Classroom (that was set to two days later considering possible grace days).

Getting Started

1. If your first name starts with A-K, you will use the host below:

   ```
   cs214-01.cs.ucr.edu
   ```

   Otherwise your host name will be:

   ```
   cs214-02.cs.ucr.edu
   ```

2. Log in to the course server by:

   ```
   ssh [your_user_id]@[your_host]
   ```

   Your user id is your UCR NetID. You will log in using your CS password.

Notes:

To get the best performance and collect reasonable final data, you may want to have all cores available (i.e., make sure you are the only one using the machine at that time). Do not wait until the last minute, since debugging and testing needs time, and also the machine might be very busy before homework deadlines. You can easily check if there are anyone else using the machine using commands `top` or `htop`.

Don't forget to back up all files you have on this machine.

It's recommended to debug locally on your own computer and only use the course server to test the performance.

Downloading Testing Code

You can find the template file of this assignment by using this invitation link:

```
https://classroom.github.com/a/JWaDJEJ0
```

You will find a sample code `reduce.h`.

When you click the link, you will automatically duplicate the repository. Then you should be able to find the instructions in the repository (in `README.md`) and shown on the page. The main steps are also shown below. I would suggest you to copy the commands/links in the version on GitHub page as some letters cannot be copied properly in pdf.

If you need to access git on the course server, you may need to set SSH key in GitHub. In particular,

1. Log into the course server as introduced above.
2. Then, generate your ssh key

   ```
   ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
   ```

3. Then set your ssh key in GitHub. First, find your public key in file:

   ```
   ~/.ssh/id_rsa.pub
   ```

   Then copy the content of this file. Go to GitHub ⇒ Your profile ⇒ SSH and GPG keys. Click "New SSH key". Paste the public key into field "Key", and give it a name you like (e.g., cs214-01 or cs214-02).

4. You are now able to access the git repository on the course server!

Run the Test Code

Then let's try to run the test code!

1. Log into the course server as introduced above.
2. Get your repository by

   ```
   git clone [ssh_link]
   ```

   Your SSH link can be found in your own repository "Code ⇒ SSH", it should start with "git@github.com:".

3. Then you can compile the code with:

   ```
   make
   ```

   and run it with:

   ```
   ./reduce [num_elements] [num_rounds]
   ```

Notes:

This sample code **will not** give you satisfactory performance. Finally in this assignment you will need to write your algorithms to get a good performance.

Answer the Following Questions

1. (0.5pt) How many cores and hyperthreads do you have in your tested machine? How large is the L1, L2, L3 cache? How did you get this information?
2. (1pts) Test the reduce code in the repository. In the following, we will use $n = 10^9$.
   (a) (0.5pt) What is the sequential running time of reducing n elements? (i.e., adding them one by one sequentially, you need to implement this on your own). Compare it with the running time of this parallel algorithm running on one core.
   (b) (0.5pt) What is the running time when you use 1, 2, 4, 8, 12, 24, 48 threads? To change the used threads, change the variable PARLAY_NUM_THREADS. For example, using

   ```
   export PARLAY_NUM_THREADS=1
   ```

to only use one thread in the computation.

3. (3.5pts) **Granularity Control.** As we mentioned in class, this version of reduce code may have unsatisfied performance, especially when you have many cores and small input sizes. This is because scheduling (forking and joining threads) causes overhead, which is significant when the input size is small. A simple trick to tackle this is to control the parallelism granularity (also called *coarsening*). When the size is small enough, we stop doing recursive calls, but directly add them up and return. We've talked about this in class. The appropriate threshold depend on the platform and the tested environment. Next you need to test the new code with granularity control, and find a good threshold.

   (a) (1pt) Describe your algorithm with granularity control.

   (b) (1pt) You may find out that the overall performance is affected by the threshold for the base case size. How to use only a few tests to find the best threshold for granularity control? Please describe your methodology to do that.

   (c) (0.5pt) Show the performance of the code with different granularity control thresholds.

   (d) (1pt) Show the best threshold and running time you get. Ideally you should get your running time down to about 0.3 seconds with 10^9 elements and all threads.